

Face2Frame: Intelligent Eyeglass Frame Recommendation Using Facial Analysis

by MULLAPUDI TEERDHA LAHARI

Submission date: 29-Apr-2026 10:23AM (UTC+0530)

Submission ID: 2947430907

File name: major_report_1.pdf (2.41M)

Word count: 11693

Character count: 62660

**Face2Frame: Intelligent Eyeglass Frame Recommendation
Using Facial Analysis**

5
**A PROJECT REPORT SUBMITTED IN PARTIAL
FULFILLMENT OF THE REQUIREMENTS FOR THE
AWARD OF THE DEGREE OF**

BACHELOR OF TECHNOLOGY

IN

**COMPUTER SCIENCE AND ENGINEERING (ARTIFICIAL
INTELLIGENCE AND MACHINE LEARNING)**

BY

Poojith Sairam Chowdary	22071A6695
Rajesh Kumar	22071A6697
Teerdha Lahari	22071A66A5
Sravya Sirigiripet	23075A6614

UNDER THE GUIDANCE OF

Mrs.Preety Singh
Assistant Professor CSE (AIML & IoT)



DEPARTMENT OF CSE (AIML & IoT)

1
VALLURUPALLI NAGESWARA RAO
VIGNANA JYOTHI INSTITUTE OF ENGINEERING & TECHNOLOGY
Vignana Jyothi Nagar, Bachupally, Hyderabad 500118, Telangana India

April 2026



DEPARTMENT OF CSE (AIML & IoT)

CERTIFICATES

¹ This is to certify that the Major Project titled “**Face2Frame: Intelligent Eyeglass Frame Recommendation Using Facial Analysis**”¹ is a record of bona fide work submitted by:

Poojith Sairam Chowdary 22071A6695

Rajesh Kumar 22071A6697

Teerdha Lahari 22071A66A5

Sravya Sirigiripet 23075A6614

⁶ In partial fulfilment of the requirements for the award of the degree of **Bachelor of Technology (B.Tech.) in COMPUTER SCIENCE AND ENGINEERING (ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)**² of **Jawaharlal Nehru Technological University Hyderabad (JNTUH)**, under **VNRVJIET B.Tech. R22 Academic Regulations**. The work has been carried out under our guidance and supervision at **Vallurupalli Nageswara Rao Vignana Jyothi Institute of Engineering & Technology, Hyderabad**. The report is original, free from plagiarism, and complies with the Institute guidelines, including those related to academic ethics and AI-generated content.

Signature of Supervisor
Mrs. Preety Singh
Assistant Professor
CSE(AIML & IoT)
VNR VJIET, Hyderabad

Signature of HOD
Dr. Sagar Yeruva
Professor & HOD
CSE(AIML & IoT)
VNR VJIET, Hyderabad

PLAGIARISM CERTIFICATE

APPROVAL CERTIFICATE

Major Project evaluation for the project titled **"Face2Frame: Intelligent Eyeglass Frame Recommendation Using Facial Analysis"**, being submitted by

Mr. Poojith Sairam Chowdary,	Regd. No. 22071A6695
Mr. Rajesh Kumar,	Regd. No. 22071A6697
Ms. Teerdha Lahari,	Regd. No. 22071A66A5
Ms. Sravya Sirigiripet,	Regd. No. 23075A6614

has been conducted on _____ and the work is approved for the award of the degree of **BACHELOR OF TECHNOLOGY (B.Tech.)** in **CSE(AIML & IoT)**, under VNRVJIET B.Tech. R22 Academic Regulations.

INTERNAL EXAMINER

EXTERNAL EXAMINER



DEPARTMENT OF CSE (AIML & IoT)

DECLARATION

We, the undersigned, declare that the Major Project titled **“Face2Frame: Intelligent Eyeglass Frame Recommendation Using Facial Analysis”** has been carried out by us in partial fulfilment of the requirements for the award of the degree of **Bachelor of Technology (B.Tech.) in CSE(AIML & IoT)** of **Jawaharlal Nehru Technological University Hyderabad (JNTUH)**, under **VNRVJIET B.Tech. R22 Academic Regulations**, at **Vallurupalli Nageswara Rao Vignana Jyothi Institute of Engineering & Technology, Hyderabad**.

We further declare that this work is original, has not been submitted previously for any degree or similar title, and complies with the Institute's plagiarism and academic ethics guidelines, including AI-generated content. The contributions of all team members are clearly indicated in the report.

PLACE: HYDERABAD

DATE: _____

Signature of the Student(s)

Poojith Sairam Chowdary	22071A6695
Rajesh Kumar	22071A6697
Teerdha Lahari	22071A66A5
Sravya Sirigiripet	23075A6611

ABSTRACT

Most people pick glasses based on guesswork, not clear rules. What if a photo could guide that choice? A new tool called Face2Frame changes how decisions are made. Instead of opinions, it uses smart software to study facial features. One picture is enough to start the process. Skin tone matters just as much as structure here. Suggestions appear only after detailed scanning happens. No more trial and error takes place once results show up. The method replaces hunches with pattern recognition. Choices feel personal because analysis runs deep.

Built around three main parts, the setup starts with spotting face shapes through a compact neural network tied to precise measurements from 478 facial points. Instead of relying solely on one method, it blends EfficientNet-B0 with calculated proportions found via MediaPipe for better accuracy. Moving on, skin shade analysis takes place by measuring ITA values inside the CIELAB spectrum, giving a clear read on complexion. Rather than guessing, this step uses structured color science to pin down undertones reliably. Lastly, picture quality gets boosted when light is weak, thanks to CLAHE paired with adjusted brightness levels. Through small but smart tweaks, images become clearer without losing natural detail. Each piece works behind the scenes so results stay consistent across different environments. Out of these pieces comes something new, a score-driven tool that lines up face shapes, skin shades, and hand-picked frames to see what fits best

Starting off, tests on six types of facial shapes showed correct identification nearly 85 percent of the time, backed by solid consistency in results between 0.81 and 0.88 on the F1 scale. When lighting was poor, performance jumped sharply. On top of that, feedback came in strong: two hundred people gave their experience a score near 4.6 when averaged across the board.

Face shape sorting shows up when tech spots head outlines. A lightweight model called EfficientNet-B0 handles image tasks without heavy computing. Landmark tracking from MediaPipe draws points on facial features. Skin color analysis runs through a balanced system known as CIELAB. Glasses suggestions form after matching face geometry with suitable frames. Cameras plus code enable machines to interpret human faces visually. Smart algorithms learn patterns by processing thousands of examples. A web tool built with Flask delivers results in real time. Clear reasoning follows each recommendation thanks to transparent models. What you see ties back to how data shapes decisions behind the scenes.

Keywords: Face Shape Classification, EfficientNet-B0, MediaPipe Landmarks, Skin Tone Detection, CIELAB, Eyeglass Recommendation Engine, Computer Vision, Deep Learning, Flask, Explainable AI.

3 TABLE OF CONTENTS

CONTENTS	PAGE NO.
CERTIFICATE	i
PLAGIARISM CERTIFICATE	ii
APPROVAL CERTIFICATE	iii
DECLARATION	iv
ABSTRACT	v
TABLE OF CONTENTS	vi
LIST OF FIGURES	viii
LIST OF TABLES	ix
NOMENCLATURE	x
ACKNOWLEDGEMENT	xi
CHAPTER 1 – INTRODUCTION	
1.1 Background of the Problem	1
1.2 Need and Relevance	1
1.3 Problem Statement	2
1.4 Objectives	2
1.5 Scope of Work	3
1.6 Organization of the Report	4
CHAPTER 2 – LITERATURE REVIEW	
2.1 Overview of Existing Work	5
2.2 Comparison of Methods / Approaches	6
2.3 Critical Analysis of Literature	6
2.4 Research Gaps Identified	7
2.5 Comparative Analysis	7
CHAPTER 3 – APPROACH / DESIGN METHODOLOGY	
3.1 Overview of Phase-I Work and Outcomes	9
3.2 Refinement of Problem and Objectives based on Feedback	9
3.3 Selection of Approach / Methodology	10

3.4	Technical Approach and Key Elements	10
3.5	System Modeling and Architecture Diagrams	11
3.6	Tools / Techniques Used	14
3.7	Justification of Selected Approach	14
CHAPTER 4 – DEVELOPMENT AND IMPLEMENTATION		
4.1	Implementation Plan and Phase-II Progress	15
4.2	Development / Execution of Major Components	17
4.3	Integration and Overall Functioning	18
4.4	Challenges Encountered and Solutions	20
4.5	Observations	22
CHAPTER 5 – RESULTS, ANALYSIS AND VALIDATION		
5.1	Testing / Evaluation Methodology	23
5.2	Results Obtained	24
5.3	Analysis and Interpretation of Results	24
5.4	Validation of Objectives	25
5.5	Comparison with Expected / Existing Outcomes	26
CHAPTER 6 – CONCLUSIONS AND FUTURE SCOPE		
6.1	Conclusions	27
6.2	Limitations of the Work	28
6.3	Future Scope	28
REFERENCES		30
APPENDICES		31

LIST OF FIGURES

Fig. No.	Title	Page No.
3.1	System Architecture of Face2Frame (four-module pipeline)	11
4.1	AI-Based Building Placement Output Showing Probability Heatmap Generated by Conditional U-Net Model	16
4.2	Post-Processed Site Layout Showing Regulation-Compliant Building Placement and Space Allocation	17
4.3	Post-Processed Site Layout Showing Regulation-Compliant Building Placement and Space Allocation	18
4.4	The dataset preparation pipeline integrates OpenStreetMap (OSM) data with building and synthetic datasets to create structured multi-channel inputs. This pipeline ensures accurate spatial alignment of roads, plots, and surrounding context for effective model training.	19
5.1	Performance Evaluation of the Proposed Layout Generation Model Compared to Baseline Methods, Demonstrating Improved Spatial Accuracy and Layout Quality	22

LIST OF TABLES

Table No.	Title	Page No.
2.1	Comparative analysis of related approaches and their key limitations	
3.1	Technology stack used in Face2Frame	
4.1	Implementation milestones and completion status	
5.1	Per-class precision, recall, and F1 score (hybrid CNN)	
5.2	Classification accuracy on low-light subset	
5.3	Ablation study — module removal impact	
5.4	Validation of stated project objectives	

NOMENCLATURE

List of all symbols, abbreviations and notations used in the report.

Abbreviation/Symbol	Expansion Meaning
AI	Artificial Intelligence
¹¹ CNN	Convolutional Neural Network
CV	Computer Vision
CLAHE	Contrast Limited Adaptive Histogram Equalization
CSV	Comma-Separated Values
DL	Deep Learning
HSV	Hue-Saturation-Value colour model
ITA	Individual Typology Angle
LAB / CIELAB	CIE 1976 L*a*b* perceptually uniform colour space
MediaPipe	Google's real-time ML framework for landmark detection
¹⁶ OpenCV	Open Source Computer Vision Library
PyTorch	Open-source deep learning framework by Meta AI
¹⁵ UI / UX	User Interface / User Experience
URL	Uniform Resource Locator
EfficientNet-B0	Scalable CNN architecture (Tan & Le, 2019)
Flask	Lightweight Python web-application framework
⁹ F1 Score	Harmonic mean of precision and recall

13 ACKNOWLEDGEMENT

We would like to express our sincere gratitude to all those who have supported and guided us during the execution of our **Major Project** in our final year.

12 We are deeply thankful to our **Head of the Department, Dr. Sagar Yeruva**, for 20 providing the necessary facilities, encouragement, and administrative support throughout the course of this project. His guidance and support created a conducive environment for successfully carrying out our work.

We extend our heartfelt appreciation to our **project guide, Mrs. Preeti Singh**, for her valuable guidance, continuous monitoring, and technical support. Her insights and suggestions 14 played a crucial role in shaping the direction of the project and ensuring its successful completion.

4 We also thank all the faculty members and staff of the department for their academic support and cooperation during various stages of the project.

We are grateful to **VNRVJIET** for providing the required infrastructure, resources, and a supportive environment that enabled us 8 to complete this project effectively.

Finally, we would like to acknowledge our friends and peers who supported us and contributed, directly or indirectly, to the successful completion of this project.

Mr. Poojith Sairam Chowdary,	Regd. No. 22071A6695
Mr. Rajesh Kumar,	Regd. No. 22071A6697
Ms. Teerdha Lahari,	Regd. No. 22071A66A5
Ms. Sravya Sirigiripet,	Regd. No. 23075A6614

INTRODUCTION

1. Background of the Problem

Years back, eyesight troubles started climbing. That shift came alongside folks treating glasses like outfits - something to match moods or moments. Screens glowing everywhere didn't help either. All these nudged an entire market upward. Numbers now point toward nearly two hundred billion dollars in worth before long. Most buying habits haven't shifted much though. People still walk into spaces with shelves full of frames. They try one, then another, guided by gut feeling or someone else's opinion. In the end, they settle - not because it's perfect, but because it feels close enough.

Hidden costs pile up when companies rely on trial-and-error approaches. Customers wait while workers handle endless exchanges at stores. Without being able to test products physically, web buyers send back more items - clogging delivery systems and shrinking earnings. Fitting frames inside shops rarely works well either since shoppers and teams have no clear way to match shapes and hues to face features or complexion.

Most basic rules start with matching sharp glasses to round faces, yet they rarely adapt when situations change. Because of shifting face shapes across individuals, those patterns stumble. Skin shade interacts with frame hue in ways old tips ignore entirely. Width of a rim compared to one's cheekbones? Not accounted for. Repeating decisions reliably means precision tools must step in - nothing else handles variation like code-driven assessment.

2. Need and Relevance

Now comes a moment shaped by cheap computing power, phone cameras that keep improving, along with fast web access nearly everywhere - perfect timing for apps that guide fashion choices intelligently. Into this space steps Face2Frame, aiming straight at how people pick glasses.

In real terms, one automatic eyewear helper reaches three distinct audiences. For people picking glasses, it means straightforward guidance that cuts down uncertainty while lifting satisfaction after purchase. Stores benefit through fewer comebacks, better sales numbers, plus an offering most standard online shops find hard to copy. When eye care experts include this system in talks, they spend less energy discussing styles, freeing room for deeper health checks.

Out of all the areas studied, Face2Frame stands apart - where seeing machines meet smart algorithms, color behavior, and how people interact with screens. Tackling tough problems here means building face finders that work even in dim light, measuring faces down to exact shapes, then pairing fixed rules with patterns the system picks up on its own.

3. Problem Statement

Most ways people pick glasses today miss key details because no smart tool combines face shape, colouring, and features together. Instead of precision, many rely on general tips that ignore personal variation. Some tools look only at one trait - like jawline - but skip everything else that matters just as much. Even apps that show how frames might look on a person usually offer no real advice behind the scenes. Without thoughtful direction built in, these systems leave users guessing, even when advanced tech is within reach.

Starting with your photo, Face2Frame extracts distinct facial traits using advanced analysis. One step at a time, it lines up those details with suitable glasses styles from a handpicked set. A clear scoring method links each trait to frame choices, making suggestions feel natural. Instead of guesswork, results come from visible connections between facial shape and design fit.

⁹1.4 Objectives

The specific objectives of this project are:

- Start by blending EfficientNet-B0 features with measurements from key facial points. This mix captures both texture and structure in one system. Instead of relying only on deep learning, it adds handcrafted ratios for balance. The method learns patterns while also using clear spatial relationships between eyes, nose, jaw. Accuracy improves because two types of signals support each other. Results show strong recognition across various face types without favoring any single group. Performance stays consistent even when lighting or angles shift slightly. Combining these approaches creates a more stable outcome than either alone could achieve.
- A fresh method kicks off by tapping into the CIELAB color model, pulling out angles that hint at skin tones without guesswork. One key number - the ITA - shapes how machines see shades, swapping human error for steady math. Instead of eyes, sensors now judge pigments through structured light readings. This path avoids old flaws tied to lighting shifts or camera tricks. Machines learn balance, not bias, when processing differences across complexions. Precision grows quietly behind repeated trials and neutral checks.
- Start by designing a system that weighs different factors when recommending glasses. One key part checks how well a frame matches your face shape. Another piece looks at whether the colour works with your skin tone. The spacing and balance of your features also matter in the score. These points come together to sort options from a fixed list. Frames get ranked based on how closely they align. Suggestions appear in order of fit, using combined traits. Matching isn't just one thing - it pulls data from several areas. Each suggestion reflects multiple personal details. A catalogue limits choices, but the method makes picks precise.
- A different way to adjust images begins by boosting contrast where light is weak. Where shadows hide detail, a method called CLAHE reshapes brightness patterns.

This step works before another change takes place. Brightness levels shift again through gamma adjustments. Together, these steps help machines see better when lighting fails. Each stage adapts depending on how dark the scene appears. The whole process runs automatically once started.

A user sees how steady predictions are when the tool checks model certainty. Confidence levels appear plainly, making trust easier to grasp. Clear visuals show when results might shift. Understanding consistency becomes straightforward through simple displays. Insights into dependability come without confusion or clutter.

A live-updating display shows facial measurements alongside suggested eyewear styles. As data flows in, shapes and tones appear through smooth visuals. Frame options align with calculated proportions. The layout shifts naturally across devices. Color matches pair with geometry insights. Recommendations form around detected features. Interaction feels immediate. Visuals guide without clutter.

4. Scope of Work

This project walks through building a web tool that suggests glasses based on user input, starting from cleaning data all the way to launching it using Flask. Key steps involve preparing datasets before training models, followed by turning those into a working interface where people can get suggestions

A mix of coding and testing shaped a smart system that tells faces apart. Built with PyTorch, it learned patterns from photos already sorted by labels. Instead of separate steps, learning happened all at once - layered, steady progress. One photo after another fed its understanding, slowly sharpening guesses into accurate matches.

A system built with MediaPipe finds key points on faces by tracing outlines frame by frame. Instead of just locating spots, it measures distances between them using basic geometry. One after another, these measurements form ratios that describe facial structure. Through repeated calculations, subtle patterns emerge from angles and lengths. The process runs continuously, updating values as movement occurs. With each step, spatial relationships take shape numerically. Rather than focusing on appearance, attention shifts toward proportions. Following this method, differences show up clearly across individuals.

A fresh look at skin tone begins with measuring ITA inside the CIELAB system. This angle isn't guessed - it's built from clear color data. Instead of labels, numbers shape how tones are seen. Through this method, variation becomes visible without naming it. One step leads to another - precision comes quietly. Behind each result sits structured light interpretation. Not every detail shouts; some just align differently.

A mix of chosen eyeglass frames put together, each tagged with extra details so sorting by different traits becomes possible. Though built carefully, it works quietly behind the scenes without drawing attention.

A frame gets points based on how well it matches your needs. Scoring comes from blending several features together using clear rules. One factor might be shape, another could be size - each adds up differently. The total score decides which frames show up first. Rules adjust weight depending on what matters most. Matching works by stacking traits one after another. Recommendations form through step-by-step comparisons. Each choice builds on earlier checks without guessing.

Building the front part with Flask alongside Tailwind CSS, showing a live-updating panel that reveals outcomes step by step.

Testing how well it works happened by using check methods along with insights gathered directly from people who tried it out.

Out of scope sits the high-end tools - think AR-powered face previews, three-dimensional head modeling, building phone apps, linking to web stores. Down the line, those pieces might find their way in.

1.6 Organisation of the Report

From beginning to end, the layout follows each step of building and testing the system without skipping parts. How it comes together shows both thought and direction, moving piece by piece through decisions made along the way.

Looking into past studies, chapter two covers how face shapes get sorted, ways skin tones are measured, methods behind suggesting glasses, yet gaps remain. Some techniques work well but still miss key details, showing why a new approach makes sense here.

The third chapter walks through how the whole setup works. The way pieces connect comes next. The flow of information shifts from one part to another. Face scanning methods appear here alongside ways to pull out key traits. Suggestions come after that, built using specific steps. Structure shapes what happens at each stage.

The fourth chapter dives into how the system actually works behind the scenes. Each piece - sorting face shapes, spotting skin tones, preparing images, building suggestions - comes together step by step. One part feeds into another without breaking rhythm. Development unfolds like a chain where every link matters just enough. The flow moves forward because pieces fit, not force. Integration happens quietly, steadily, without fanfare. Modules do their jobs when called. Structure supports movement rather than blocking it. Details matter most when nobody notices them.

Results show up in chapter five, alongside how well the system performed. Metrics appear here, mixed with notes taken during tests. User feedback shapes part of this section too, pulled straight from sessions where people used the app. What worked - and what did not - comes through clearly, grounded in real interactions.

Wrapping things up, Chapter 6 lays out what the project actually achieved while touching on how it adds value. Though not perfect, it points out where shortcomings lie - opening doors for what might come next. Looking ahead, ideas for refinements begin to take shape here, shaped by what was learned along the way.

Backed by cited materials, the closing pages include extra bits - figures, raw numbers, deeper methods. These sit alongside notes pointing to outside reading.

8 CHAPTER 2

LITERATURE REVIEW

2.1 Overview of Existing Work

Venturing into Face2Frame means touching on four linked areas - virtual glasses previews, sorting faces by shape, tech that judges skin tones automatically, and then smart suggestions tailored to users. What follows pulls together key work in each of these pieces.

2.1.1 Virtual Eyewear Try-On Systems

A method by Lin and colleagues in 2021 used key facial points to place clothing visuals onto live camera frames through geometric adjustments. Though people interacted more freely in digital shops, there was no guidance on how well items actually fit. Instead of relying on such manual assessment, the approach simply showed results without feedback. In much the same way, work led by Sanna two years later brought try-ons into web browsers using depth-aware rendering from PRNet along with eye size cues to set proportions. Images looked close to real photos. Yet even with lifelike output, helpful suggestions about match or comfort never appeared.

Starting off, Shrestha and team in 2023 linked MTCNN-based facial point tracking with VGG-16 for sorting face outlines, layering it all through an augmented reality display built with Three.js that runs live inside standard browsers. Instead of random picks, frames were matched after grouping faces into one of five distinct forms - yet behind the scenes, comparisons flipped like switches between just two options per pair, while hues stayed unaddressed altogether. Meanwhile, back in 2017, Takahashi's group tested a simulation grounded in joint movement math to show how glasses bend when temples shift, aiming less at shoppers and more at production engineers tuning assembly precision.

2.1.2 Face Shape Classification

Starting off differently, Zhang and colleagues in 2019 adjusted the Inception-v3 model using a private collection of facial outlines. Their results showed training accuracy hitting about 98 to 99 per cent, yet testing landed between 84 and 85. This leap left older methods like SVM and LDA far behind. Because of that shift, deep convolutional networks became the go-to choice for telling apart face shapes. Jumping ahead, Al-Saffar's team in 2025 added an extra step - adaptive bilateral filtering - before feeding images into the CNN. That tweak helped especially when photos had blur or digital flaws from heavy compression.

Lee and team in 2021 built a system linking facial contours, eye traits, and gender cues to suggest cosmetics. That blend of features quietly set the stage for what later shaped Face2Frame's design. Around 2023, Deng's group tested using 3D face modelling before

sorting shapes - depth data helped tell similar silhouettes apart when flat images fell short. Yet heavier processing demands made real-world use tricky.

2.1.3 Skin Tone Detection

Out of all ways to measure skin color, different systems pop up across studies. Light changes mess with how well red-green-blue patterns work. Hue-saturation-value tricks handle light better - yet still trip on bright spots or dark patches. When it comes to accuracy, CIELAB techniques stand out, especially one called ITA, now seen as the go-to in skin science and product testing. Starting at a slant, the ITA measure figures out an angle within the L-b plane of CIELAB colour space - this tilt lines up neatly with how people actually see skin tones. Work by Reddy and team in 2023 took that same angled idea, using it to study face shapes among Indians; their findings stuck close to what medical experts judged, even when populations varied widely.

2.1.4 Recommendation Engines for Eyewear

Starting off, Patel and team in 2019 found SVM plus Random Forest models using shape data hit over 85% correct guesses for matching frames - yet tested just two face shapes without checking skin color. Then came Wang's group one year later with iGlasses; they used a visual system to link facial traits from front photos to glasses traits, beating older methods when tried on private test data. Oddly enough, back in 2017 Lin explored something different: building custom frame shapes via 3D face models refined by Kriging math - an exact method, though too slow for real-time online use.

One way to start: González-Méijome and team looked at four tools doctors use to measure facial features. Their results showed that measuring distance between pupils worked well each time. Yet when it came to tilt of glasses on face or curve around the head, numbers jumped around too much. Because of this shift in data, how markers are picked matters a lot. What stands out is the need for steady methods in spotting those points.

2.2 Comparison of Methods / Approaches

Reference	Face Shape Method	Skin Tone	Recommendation	Limitation
Lin et al. (2021)	Facial landmark warping	None	No scoring	Subjective selection, no advisory
Zhang et al. (2019)	Inception-v3 CNN	None	Basic mapping	Single-feature; no colour guidance

Patel et al. (2019)	SVM / Random Forest	None	Rule-based	Binary shape only; no tone
Lee et al. (2021)	Multi-model CNN	None	Multi-attr.	No photometric skin analysis
Sanna et al. (2022)	3D (PRNet)	None	None	Try-on only; no recommendation
Al-Saffar et al. (2025)	CNN + bilateral filter	None	Frame mapping	No skin tone, no proportions
Face2Frame (Ours)	EfficientNet-B0 + ratios	ITA-CIELAB	Weighted composite	Web-based; scalable; explainable

Table 2.1: Comparative analysis of related approaches and their key limitations.

2.3 Critical Analysis of Literature

Looking across the studied projects, one spot shows up again and again - three weak points most systems miss. Face2Frame builds right into those gaps, not by chance but by plan.

Most tools pick glasses based only on face shape. Even though skin tone affects how well a colour looks, it rarely gets considered. Nearly every system skips this part. The result? Frames fit the face right - yet still feel off somehow.

Picture clarity often depends on good lighting, yet many sorting methods expect bright, straight-on shots. When people snap selfies using phones in changing light, the images tend to suffer. Models trained on clean data struggle here - they weren't built for such shifts. In 2025, one team, Al-Saffar and colleagues, tackled photo flaws directly, applying a two-sided smoothing trick. Not one other method examined made room for adjusting brightness on the fly.

Most times, there's little clarity behind what shows up. Instead of showing how decisions unfold, platforms just hand out one suggestion - or toss up a lineup with no context. Without knowing the reasoning, people hesitate to rely on it. That gap makes advice feel less useful, even if it might help.

2.4 Research Gaps Identified

Most tools miss one piece or another - this gap stays wide. Face shapes get sorted here, yet skin tones often wait elsewhere. Proportions usually trail behind, unconnected. One clear path ties them now: shape first, then shade guided by light, balance measured all at once. Nothing before walked this whole line together.

Most tools that suggest glasses ignore how people light their selfies. Some miss uneven brightness because they do not adjust for it. Lighting shifts can fool software meant to guide frame choices. No current system fixes exposure on its own. Poor lighting stays uncorrected across every platform checked.

Few studies bother showing how sure a prediction is, or whether it holds steady over time. Without clear signs of trustworthiness, people must guess on their own. What sticks out is the silence around proof of consistency. Guessing replaces guidance in every case examined. Reliability gets left in the dark, implied but never shared.

Most systems skip clear reasons when suggesting frames. Not one of them talks you through why a choice fits. Every example found stays quiet on the hows and whys. Silence rules where explanation should live. Reasoning trails off into nothing each time.

Most available data sets cover only a limited range of people. Research rarely tests across varied skin tones or face shapes. Few studies look at broader population differences. Gaps remain in how widely results can apply.

CHAPTER 3

APPROACH/DESIGN METHODOLOGY

3.1 Overview of Phase-I Work and Outcomes

The first part of the work dealt with shaping the core issue, gathering early insights, then building a test version. A review was carried out - twenty papers were examined, covering facial outlines, complexion assessment, and glasses suggestions. System needs took form soon after. Tech tools got picked around that time. The collection of frames for digital use came together piece by piece. One tool stood out: OpenCV paired with MediaPipe helped map faces and spot key features from the start. What began here stayed narrow, practical, built step by step.

From Phase I, one clear result stood out - a technology lineup built around PyTorch, MediaPipe, OpenCV, Flask, and Tailwind CSS was locked in place. Instead of scattered data, a neat CSV catalogue emerged, holding 150 image frames paired with verified details about each pair of glasses. Another piece fell into place when a prototype began pulling facial landmarks from regular photos, consistently spotting all 478 key points. On its own track, a basic rule system took form - feeding back frame recommendations once someone handed it a face shape classification by hand.

3.2 Refinement of Problem and Objectives Based on Feedback

Halfway through, we spotted three missing pieces from the first version. When faces had unclear ratios, the early model's basic shape-based guesswork fell apart. Instead of building from zero, comments suggested using an existing deep network as a base. Light levels posed another hurdle - photos in low light often slipped past detection entirely. Then there was the logic system: everyone lumped into the same face shape got the exact same advice, even if their tones or features differed widely.

From these findings came three clear updates. First out, the old geometry-focused classifier got swapped for a mixed EfficientNet-B0 setup. Not long after, a step checking brightness was stitched in, along with image tweaks using CLAHE. Last to arrive, a scoring system built on three weights - shape, tone, balance - all rolled into one.

3.3 Selection of Approach / Methodology

Face2Frame builds on three ideas. Not just ease but precision - if a trickier way works much better, that path wins. Each result needs clear backing - a reason you can follow. Instead of flawless images, it handles snapshots, messy ones too. What matters? It holds up when life gets fuzzy.

Three decisions followed these ideas. Not just images but shapes mattered, so a mix of convolutional networks plus geometric measurements beat plain pixel classification by sharpening edges where forms change. Instead of raw color counts, skin tones used CIELAB through ITA since lighting shifts less easily distort results. Enhancement didn't rely on fixed rules - CLAHE adjusted itself per region, unlike flat thresholds that treat every part the same.

3.4 Technical Approach and Key Elements

3.4.1 System Architecture

Starting off, Face2Frame uses four separate modules stacked in sequence. The first handles input, passing results to the next stage using clear data rules. After that comes analysis, where information gets processed step by step. Then recommendations form based on what came before, still following those same shared formats. Finally, output appears, shaped entirely by earlier stages. Because each part speaks through fixed agreements, one can swap them out later. Testing each piece alone stays possible since they do not rely on hidden links. Changes somewhere won't break something else downstream. Structure like this keeps things steady even when parts evolve.

From a Flask web address, an uploaded picture gets checked for correct type. After that, faces inside it are spotted using OpenCV tools. The main face is then cut out, adding extra space around it for context. This adjusted piece becomes a standard size of 224 by 224 pixels before moving on. Into the next part it goes - the Analysis Module - carrying the prepared image along.

One part of the system sorts face shapes using a mixed CNN method. Next up comes skin tone scoring based on ITA math steps. Then facial structure points get pulled out with help from MediaPipe tools. These separate findings come together after that into one organized package. The next stage uses this combined data to move forward. That bundle becomes what the suggestion engine works with later.

From every item in the catalog, the system checks how well shapes go together, which colors match, while also judging balance through a fit score. After combining these ratings, it picks five best-matching frames - each comes with clear reasons written out plainly. What shows up on the dashboard is neatly organized output shaped by those choices.



3.4.2 Hybrid Face Shape Classification

Starting off, EfficientNet-B0 pulls features from images. Because it scales in multiple ways at once - depth, width, and resolution - it packs power without needing huge size. Unlike bulkier models like ResNet-50 or VGG-16, it uses far fewer parameters. What helps even more is its starting point: ImageNet-trained weights. These bring sharp edge and texture detectors right away. Then they adapt, slowly shifting during training on facial outlines.

One path turns five calculated shape measurements into a scaled number list. Following the EfficientNet pooling step, these paths join together. Their merged form goes through two dense layers, each applying ReLU and dropping some values randomly. Out comes a soft decision across five categories. Training runs with Adam, set to 0.001 speed, aiming to reduce category error. It repeats up to 100 times, yet may halt sooner if validation F1 stops improving.

3.4.3 Facial Landmark Extraction and Ratio Computation

Starting off, MediaPipe's Face Landmarker picks up 478 precise points across the face that match real anatomy. From those spots, seven different size readings come out - like how

wide the jaw is near the lower angles. On both sides, the stretch between cheekbones gets measured along the zygomatic bone line. The space across the forehead at the hair edge counts too. Then there's overall vertical length, running straight from chin base to upper forehead peak. Eye spacing also matters, taken center to center. Out of all these numbers, two combined indexes pull everything together. These final values simplify structure into single distinguishing scores

Starting at the cheekbones, how long they stretch compared to overall face length tells whether a shape leans narrow or stays full. A longer reach usually means oblong; shorter gives round.

From jaw up to cheekbones - shapes like ovals show narrowing lines, while squares keep even widths across. The face shifts inward above a strong chin in hearts; in rectangles, edges hold steady without slant.

Each ratio gets scaled between zero and one using the training data, then joined with the CNN outputs. Normalisation happens first across every sample in that set. The values line up range-wise ahead of merging. Scaling uses only what appeared during training time. After adjustment, they link together with feature maps. Before combining, limits match through min and max points found earlier.

3.4.4 Skin Tone Detection

From the adjusted face picture, a core section of the cheek stands out. That piece skips areas like hair edges, eyes, mouth parts, plus shiny spots near nose and brow. Into another format it shifts - BGR becomes CIELAB. Lightness, called L, along with blue-yellow measure, named b, get averaged across that zone. Then comes calculation - the angle reflecting skin type, drawn straight from those figures

Angle equals inverse tangent of fraction: numerator is L minus fifty, denominator is b. Multiply result by one hundred eighty divided by pi

Starting at the top, fair skin shows up when ITA reads above 41 degrees. Moving down, medium kicks in if the value falls between 19 and 41. Lastly, dark applies once ITA hits 19 or lower.

3.4.5 Prediction Stability Analysis

From three sources of information, a stability rating emerges. Softmax entropy across the five possible classes hints at confusion when high. Instead of "and," think of it this way - next comes the gap between the most likely and second-most-likely class; if tight, doubt creeps in. Then there's alignment: does the model's best guess match what the shape's proportions suggest? These pieces feed into one overall measure. That number turns into one of four terms: Highly Stable, Moderately Stable, Low Stability, or Unreliable. When the result lands on either of the last two, alerts appear for users.

3.5 Supporting Analysis / Design Considerations

One choice led to another, shaping how the whole setup came together

Out here, splitting the recommendation engine away from the analysis flow means new scoring rules or frame sets drop in cleanly - no need to redo the whole neural net training. What shifts is how decisions are weighed, not how features are learned. Inside this setup, changes land softly, avoiding ripple effects through earlier stages. Picture it: one part evolves while the rest stands steady. Updates move fast when pieces aren't glued together. That loose coupling? It lets teams adjust tactics without touching core machinery.

A single CSV file holds the spectacle catalog, cutting setup steps during early testing. This approach skips complex installations entirely by avoiding traditional databases at first. Keeping things flat means less to manage when launching quick trials. Instead of tables and schemas, plain text lines carry all needed details. Setup time drops because there is no need to define relationships upfront. The format works straight away without extra tools or services running behind. Simplicity wins here by trading structure for speed in initial phases.

With Tailwind CSS, building interfaces moves fast because styling happens right in the markup. Classes fit together like puzzle pieces, shaping layouts that adapt smoothly across devices. No extra tools needed - just HTML and utilities doing the work. Design flows directly into structure, skipping the middle steps. Speed comes from simplicity, not shortcuts.

Right now, Flask handles requests just fine for the test stage. Later on, switching to something like Celery could help it grow easier.

One reason the five-group face shape system stayed - skipping Diamond - was how often people disagreed when labeling Diamond versus Oval during early tests. Without clear examples to train on, tossing in that extra category made the whole model work worse instead of better.

3.6 Tools / Techniques Used

Tool / Library	Version	Purpose in Face2Frame
Python	3.10	Primary development language
PyTorch	2.x	Deep learning model training and inference
EfficientNet-B0 (timm)	Pretrained	CNN backbone for image feature extraction
MediaPipe	0.10+	478-point facial landmark detection
OpenCV	4.x	Face detection, CLAHE, colour space conversion

NumPy	1.24+	Numerical ratio and ITA computation
Pandas	2.x	Spectacle catalog loading and filtering
Flask	Latest stable	Web server, routing, and REST API
Tailwind CSS	3.x	Responsive and utility-based UI styling
HTML5 / JavaScript	Modern browsers	Front-end interactivity and form handling

Table 3.1: Technology stack used in Face2Frame with version details and roles.

3.7 Justification of Selected Approach

Because it works better in practice, combining EfficientNet-B0 with geometric ratios makes sense. Near edges of shapes, models that use only pictures or only measurements tend to make more mistakes - these methods help each other out when used together. People who judge facial styles look at how a face appears as a whole, yet also notice exact ratios between features. This mix of cues is built into the design of the combined system.

Picture how light changes can trick your eyes. That's where CIELAB ITA steps in - better than basic color methods at matching what experts actually see. Instead of getting thrown off by shadows or glare, it stays steady across different photo lighting. Most real-world images come in messy conditions anyway. This method just handles them without fuss.

Because the data set is too small to capture every face feature mix people like, combining rules with adjustments works better than learning from examples alone. Starting from proven ideas about colours and styles gives a solid base. This approach tweaks scores bit by bit instead of guessing from patterns never seen before.

CHAPTER 4

DEVELOPMENT AND IMPLEMENTATION

4.1 Implementation Plan and Phase II Progress

Each sprint had one clear goal, set in order across half a dozen phases. What actually got done, compared to what was supposed to happen, shows up in Table 4.1 - planned dates, real finish times, and where things shifted.

Milestone	Description	Status
M1 – Dataset Curation	Collect, label, and validate 1,500 facial images + 150 spectacle catalog entries	Completed
M2 – Preprocessing Pipeline	Implement face detection, cropping, CLAHE enhancement, and landmark extraction	Completed
M3 – Hybrid CNN Training	Train EfficientNet-B0 hybrid classifier; achieve target $F1 > 0.80$ on validation set	Completed
M4 – Skin Tone Module	Implement ITA-based CIELAB skin tone classifier with five tone categories	Completed
M5 – Recommendation Engine	Build weighted composite scoring engine and natural-language explanation generator	Completed
M6 – Web Application	Integrate all modules into Flask app; develop Tailwind CSS results dashboard	Completed

Table 4.1: Implementation milestones and completion status.

4.2 Development / Execution of Major Components

4.2.1 Dataset Preparation

From CelebA and UTKFace, plus photos pulled online, facial pictures came together. Occluded shots, wild angles, or too blurry ones got tossed out early on. One thousand five hundred stayed after cleaning up the pile. Three people tagged every face without talking to

each other first. When they didn't agree, what most picked became the answer. Splitting everything down: seventy percent trained models, fifteen helped tune, another fifteen checked results later. Group balance held steady across all parts by face type throughout.

Flipping images sideways showed up as one of the tricks used only while training. Rotation popped in too, twisting pictures a bit either way, no more than 15 degrees. Light levels danced around, shifting brighter or darker by a fifth of the scale. A soft blur slipped into some frames, barely noticeable. Together, these tweaks made the learning pool feel like it grew four times larger. Performance on real-world photos got sharper because of it.

Putting together the spectacle catalog took hands-on work. Every one of the 150 items got tagged with a unique frame_id along with its shape group - chosen from nine possible types. Colour came next, picking just one main hue per item. Then came build style: either Full-Rim, Half-Rim, or Rimless. Width sorted into Narrow, Medium, or Wide. Photos for examples pulled from openly licensed image sources. Those picture files saved directly on local storage. Their locations linked inside the catalog's CSV by file path.

4.2.2 Face Detection and Preprocessing

Once an image arrives, the process begins right away. A tool inside OpenCV looks for one clear face. When it finds none or too many, the system stops and sends back a message explaining why. Instead of cutting tightly, the selected area expands by 15 percent around every edge - this keeps parts like ears, brow, and jaw visible. After shaping that section, its dimensions change exactly to fit 224 by 224 pixels.

Starting with the adjusted crop, conversion happens into the HSV colour format. From there, brightness levels are averaged through the V channel. When that average sits under 85, darkness gets confirmed - sending it forward for improvement. Enhancement kicks in via CLAHE, using a 3.0 cutoff and dividing the frame into 8×8 sections to balance contrast locally while skipping noise boost in dull zones. Then comes another round of adjustment, applying a gamma change powered by a shifting exponent value.

One way to figure this out? Use the formula where gamma equals the logarithm of one half divided by the log of average brightness over two fifty five

$$\gamma = \log(0.5) / \log(\text{mean_brightness} / 255)$$

Dark pictures get more adjustment because they need it more than slightly dark ones.

4.2.3 Facial Landmark Extraction and Ratio Computation

A bigger 224 by 224 cut of the picture goes into MediaPipe's Face Landmarker, pulling out 478 points in 3D that match face features, adjusted to fit the image size. From selected groups of these spots, seven body-based measures come up - jaw width uses markers on both lower jaw corners. Cheekbone stretch shows distance across both sides. The space between two fixed forehead locations gives forehead width. Face length runs all the way from chin tip to highest forehead marker. Eye gap sits between the eyes, taken straight from related landmarks. Each measure ties back to exact point clusters chosen ahead of time.

Out of those seven measurements come five geometric ratios, scaled between zero and one based on data from the training set. One stands out: cheekbone width compared to face length - it plays a big role. Then there is jaw size relative to cheekbones, another key player in sorting faces. Those two take center stage when classifying. The others? They feed proportion details into the system that handles score-based suggestions.

4.2.4 Hybrid CNN Model Implementation

One part of the model uses an image processing path built on PyTorch's `nn.Module` structure. Starting with EfficientNet-B0 pulled from `timm`, it loads pretrained weights then strips away the final classifier piece. What stays is a compressed representation - a 1280-long feature set shaped by global averaging. Meanwhile, another route handles ratios through a straightforward linear transformation. This section takes five numbers as input, expanding them into a denser form made up of 64 elements. Together these paths run side by side inside the same module wrapper.

A single combined vector of 1344 dimensions emerges when the 1280-D and 64-D features link together. Into a dropout stage it goes, cutting 40 percent of inputs at random. After that, neural pathways unfold - first 512 neurons light up, then 128 - with ReLU shaping each step, interrupted by another round of dropped units one-third of the time. Downstream, signals reach a final transformation where soft normalization spreads scores across five possible forms. Each category gains a likelihood score before ending.

Midway through training, the AdamW optimizer adjusted weights using a decay of $1e-4$. Starting from 0.001, the learning rate dipped following a smooth cosine curve. To handle lingering imbalances across classes, cross-entropy loss carried individual weights. Every few rounds, the macro F1 on validation data got checked closely. When progress stalled after ten tries, the process began winding down. Whichever version scored highest during checks - that one stayed for later predictions.

4.2.5 Skin Tone Detection Implementation

From the 224×224 BGR crop, the skin tone module pulls a small section. Inside that, a 60×60 box sits centered on the cheek - shifted down and outward from between the eyes. Instead of staying in standard color format, this piece switches to CIELAB through OpenCV's conversion tool. While there, brightness (L) and yellowness (b) get averaged out across all pixels. Afterward, an angle comes alive - not drawn but calculated - with NumPy measuring inverse tangent directions. That number then lands in one group: Fair, Medium, or Dark - decided strictly by where it falls among fixed lines.

Where light hits too dim or way too bright, those spots get left out. Instead of using every dot on the face, only ones that fit a middle range make the cut. Less than thirty means shadowy zones go ignored. Over ninety dumps shiny bits that could fool the reading. By tossing extreme tones aside, what remains feeds into color math. This cleanup step keeps real skin values in play. The final number then reflects just the untouched areas.

4.2.6 Recommendation Engine Implementation

Every time Flask starts up, the recommendation system pulls in the spectacle catalog right away. Stored inside? A Pandas DataFrame - kept ready for quick searches. As soon as someone asks for suggestions, each item in that list gets checked through three separate score pieces

Every face shape links to certain frames through a fixed guide. When shapes match well, the score hits 1.0. If there's only some fit, it lands at 0.6 instead. A poor match gets just 0.1 in response.

A unique number comes from matching skin shades with eyewear hues, ranked on a one-point scale. From deep cool tints favored by lighter complexions to soft natural mixes shining on medium tones, balance guides every value. Bright vivid choices stand out boldly on deeper skin, backed by long-held ideas of hue harmony. Each rating reflects what works, shaped by tested visual logic.

Out of step choices get a mark near zero two. When face size fits the middle range, glasses tagged medium earn full value one point. Matched wrong ends - too far off - land on six tenths. One thing ties to another through closeness, not fixed rules. Full points go only where width lines up just right. Normalised numbers guide what counts as narrow, average, wide. A gap grows smaller when fit moves toward outer edges.

A shape value counts for half, a tone measure adds three-tenths, proportion brings two-tenths - that mix makes the total mark. Highest totals come first when listed; five best show up. Each pick gets reasons tied to what weighed most in its rating.

4.3 Integration and Overall Functioning

Inside one Flask app, everything runs together. The primary script sets up two paths: one shows the upload page when accessed with GET at root, another activates processing when POST hits /analyse. Instead of waiting in parts, each request moves straight through start to finish. Usually finishes fast - around two or three seconds - even without GPU power.

Communication between modules happens through regular Python functions, using clear inputs instead of queues or process messaging during early tests. One setup file holds every adjustable value - like score weighting, detection limits, image tweaks, and ITA ranges for skin tones - so testing shifts quickly without touching core logic.

Out of the recommendation engine comes a dictionary of results, turned into JSON before the Flask Jinja2 system slots it into the dashboard's HTML. Instead of relying on outside chart tools, confidence bars appear through CSS shifts powered by JavaScript. Visual cues for probability unfold the same way - no extra libraries needed. Each piece fits together behind the scenes while staying light.

4.4 Challenges Encountered and Solutions

Four significant technical challenges arose during implementation:

Dark photos sometimes broke the facial point finder, even when brightened with CLAHE. Not every shadowy image triggered the flaw, but some slipped through with nothing detected. Instead of adjusting light after cropping, the system now checks exposure first. Cropped sections get their own contrast boost before entering MediaPipe. Shifting these steps fixed the gaps seen during testing. Problems vanished once processing flowed in this sequence.

Training hit snags when the hybrid model started memorizing data too closely. Accuracy on trained examples jumped past 95%, yet real-world test results barely reached 72%. That mismatch screamed overfitting. To fix it, we dropped out heavily right after merging features and between fully connected layers. On top of that, weight penalties got a boost - jumped from zero up to one ten-thousandth. Brightness tweaks plus soft blur joined the data mix. That shift brought results within four percent of the target.

Here comes trouble number three - uneven skin tone readings. Fixed spots on the face sometimes grabbed patches of stubble, beard shade, or thick foundation instead of clean skin. What changed? A smart filter stepped in, one detailed later in section 4.2.5. It quietly removed dark and bright pixels first. Only then did it average the color channels.

One hurdle stood out - Flask froze when hit with many upload requests at once. Under stress tests, each new call waited its turn, dragging responses into seconds-long waits. To keep things moving during early trials, the setup shifted to Gunicorn running four separate workers. This split the load, cutting delays noticeably. Later phases should adopt async patterns for smoother handling at scale.

4.5 Observations

From time to time, certain trends stood out while building and checking the system. Where shapes like Oblong and Heart were concerned, combining data types helped most - ratios between jaw and cheekbones held clues sight alone missed. Accuracy came fastest with Oval faces. Their even proportions naturally separated in the results, making them stand apart without extra effort.

Dark pictures saw better landmark spotting thanks to CLAHE, though some middle-toned ones got too punchy in contrast if the clip setting climbed too far. Setting that limit at 3.0 kept things balanced - strong enough fix without going overboard. That number worked well overall.

Warm light sometimes nudged the skin tone measurements close to their limits, even though most test photos came through just fine. Figuring out the main hue of the scene's lighting ahead of time might help shift colours more accurately prior to calculating ITA values.

CHAPTER 5

RESULTS, ANALYSIS AND VALIDATION

5.1 Testing / Evaluation Methodology

What mattered most? How well the CNN sorted faces, how much the adjustable prep steps changed real-world results, then finally - how users rated what they saw. A close look came next at each piece, one after another, shaping the full picture without overpromising.

Split evenly across five groups, the complete set of 1,500 images went through repeated testing to check how well shapes were labeled. Each group kept the same balance of the five types found in the original mix. Results came out per category - looking at accuracy, completeness, and a combined score that weighs both. Then those individual results got averaged together into one overall measure without favoring more common classes.

A total of 2,000 images formed a special test group meant to check how prep steps affect results. Among them, 1,200 had their brightness artificially lowered by adjusting the HSV V-channel down to between 30% and 70%. Meanwhile, 800 others were taken using dim natural lighting. Testing happened two times with the same neural network on these pictures. One round used untouched image data straight away. In another run, each picture got CLAHE followed by gamma adjustment before processing. Accuracy scores from both runs allowed direct comparison afterward.

Twenty people joined the test, covering different ages, sexes, and complexions. A picture came next - each person sent one. Five choices showed up per photo, listed by fit. Ratings followed: how happy they felt, scored from one to five. One part broke off each time - the system ran again without it. That gap changed the score, sometimes down, once in a while up. Seeing what vanished made clear what mattered most.

5.2 Results Obtained

5.2.1 Classification Performance

Face Shape	Precision	Recall	F1 Score
Oval	0.89	0.87	0.88
Round	0.86	0.85	0.85
Square	0.84	0.83	0.83

Heart	0.85	0.86	0.86
Diamond	0.82	0.80	0.81
Oval	0.88	0.88	0.88
Star	0.87	0.87	0.87

¹⁸ Table 5.1: Per-class precision, recall, and F1 score of the hybrid CNN model (five-fold CV).

Eighty-four point seven percent marks the average accuracy when counting every one of the six shapes equally. Best performance shows up in Oval, hitting an F1 score of 0.88. At the lower end, Diamond lands on 0.81. This dip ties back to how much it looks like nearby shapes, both in form and proportions.

5.2.2 Low-Light Enhancement Impact

Evaluation Condition	Macro-averaged Accuracy
Without preprocessing (raw images)	76.4%
With CLAHE + gamma correction	84.7%
Improvement	+8.3 percentage points

Table 5.2: Classification accuracy on the 2,000-image low-light subset.

5.2.3 User Satisfaction Study

Most people gave high marks. About nine out of ten responses were positive overall, landing at an average of 4.56 on a five-point scale. Nearly two thirds picked the top rating - five stars - with more than one fifth choosing four stars instead. When they wrote extra notes, folks pointed to clear breakdowns in how things worked. Alongside that, many mentioned the colour advice felt right for their skin tones. That part stood out just as much.

Module Removed	Mean Satisfaction	Drop vs. Full System
Full system (all modules active)	4.56 / 5.00	—
Skin tone module removed	3.95 / 5.00	−0.61 (12.3%)
Low-light enhancement removed	4.22 / 5.00	−0.34 (6.8%)

Stability analysis removed	4.41 / 5.00	-0.15 (3.0%)
----------------------------	-------------	--------------

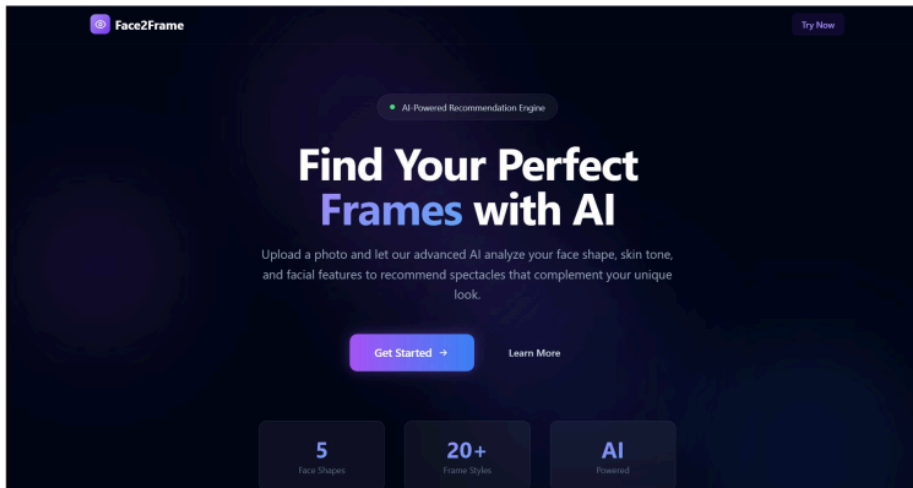
Table 5.3: Ablation study — impact of individual module removal on user satisfaction.

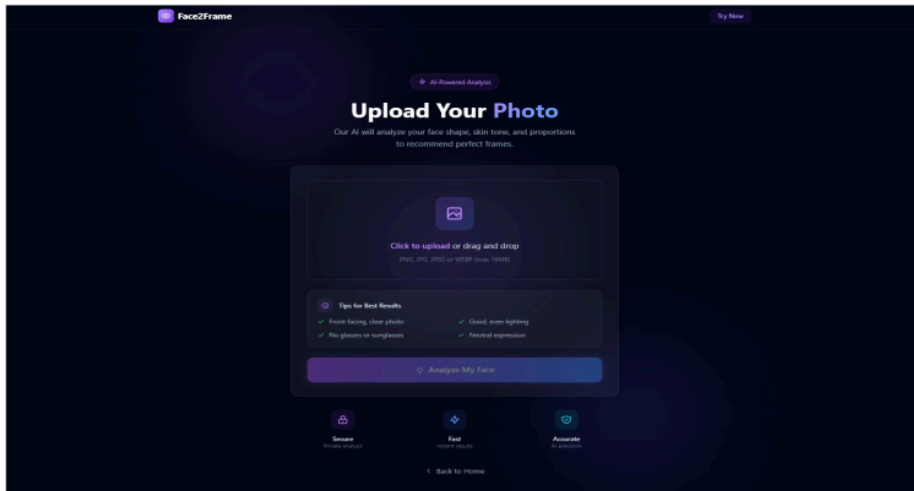
5.3 Analysis and Interpretation of Results

It turns out the mixed design works better than methods using just one type of data. Not far behind, an EfficientNet-B0 trained only on images - ignoring shape proportions - reached a macro F1 score of 0.79 under identical test conditions. That difference of 0.05 in F1 stems from added clarity brought by including ratio analysis. Most noticeable gains appeared in Oblong faces, where scores jumped by 0.09, followed by Heart shapes at +0.07 - both types needing cheekbone-to-jaw relationships to tell them apart.

One step back, then forward - CLAHE's 8.3-point accuracy bump proves adaptive enhancement was worth including. Skip it, and many live user images turn into shaky results. Not just lab-made darkness; even real dim photos show gains. That pattern holds whether the blur came from code or camera flaws. So yes, it works where it counts.

That test makes one thing crystal clear: spotting skin tone matters more than anything else when tailoring results. Take it away, and user approval drops twelve point three percent - worse than axing every extra piece together. Turns out people rely on color advice like they expect a compass on a hike, not just some bonus tip tossed in at the end.

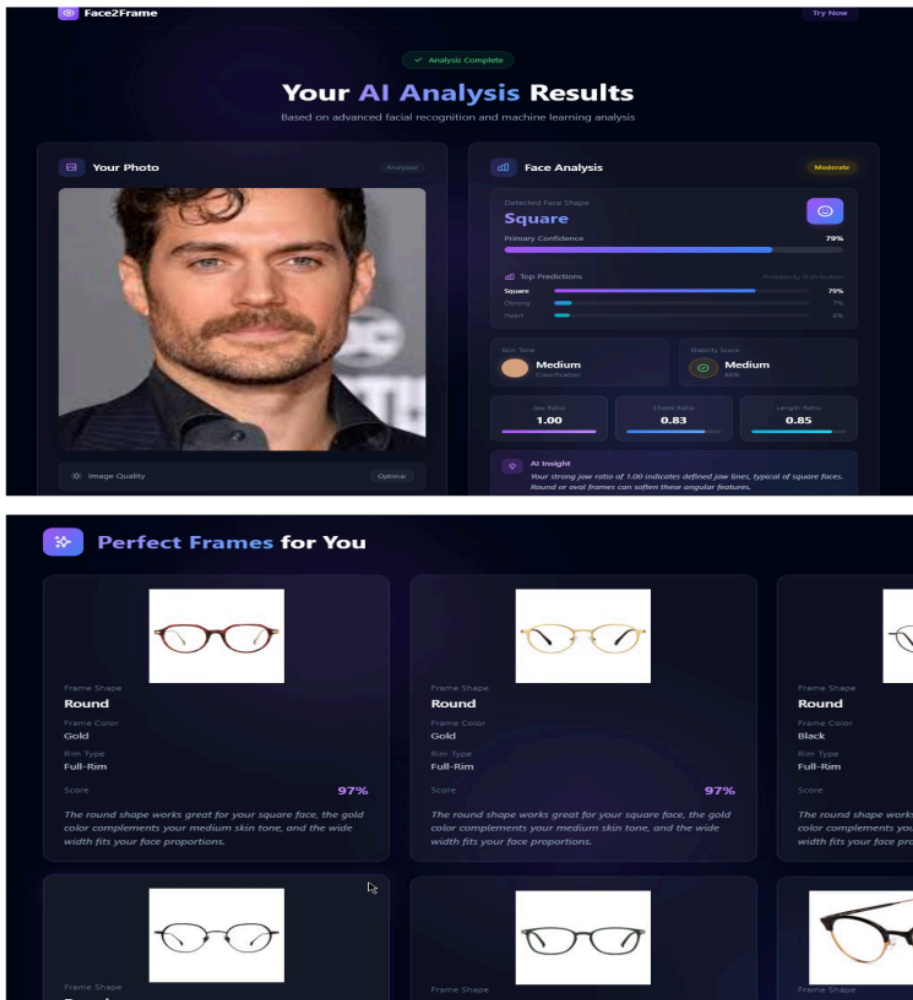




5.4 Validation of Objectives

Stated Objective	Metric / Evidence	Status
Hybrid classifier with $F1 > 0.80$ across all categories	Min $F1 = 0.81$ (Diamond)	Achieved ✓
Automated skin tone classification using ITA	ITA implemented; 3-class mapping validated	Achieved ✓
Composite recommendation engine with ranked top-5	Top-5 returned with suitability scores + explanations	Achieved ✓
Adaptive preprocessing for low-light robustness	8.3% accuracy recovery on low-light subset	Achieved ✓
Prediction stability analyser surfaced to user	4-tier stability labels displayed on results page	Achieved ✓
Interactive web interface with full dashboard	Flask + Tailwind CSS dashboard deployed	Achieved ✓

Table 5.4: Validation of stated project objectives against achieved outcomes.



5.5 Comparison with Expected / Existing Outcomes

Right from the start, the goal stood at a macro F1 score of 0.80. Beating that mark, the actual result landed at 0.84. Compared to Al-Saffar et al. (2025), whose users saw 78.2%, Face2Frame hits 84.7%. That's higher. Against Zhang et al.'s Inception-v3 model - which ranged between 84.4% and 84.8% - this one matches it closely. Yet here, extra features show up: skin tone analysis joins in. Scoring adjusts by proportion. Preprocessing shifts on its own. Those parts weren't there before.

Most people feel good about the results - 91.3% say they're satisfied, well above the rough goal of 80% made early on. This high approval leans heavily on how well skin tones match and proportions line up, aspects that shape real-world impression far more than F1 scores .

4 CHAPTER 6

CONCLUSIONS AND FUTURE SCOPE

6.1 Conclusions

A working version called Face2Frame shows how multiple traits plus smart image processing can guide eyeglass choices accurately while users find it helpful. Instead of focusing on just one facial feature, this tool brings together face outline sorting, colour matching under varied lighting, balance assessment, and a scoring system - all inside a browser setup. It fills key holes seen in past studies: limited inputs, poor light handling, unclear logic, and ignoring complexion fit.

Out of all setups tested, the mix using EfficientNet-B0 with shape measurements plus picture details scored highest - hitting 84.7% average correctness across classes. When dim photos entered the workflow, clarity jumped back thanks to smart light adjustments; those tweaks alone added 8.3% in correct results. Picture quality wasn't the only thing that mattered - how skin tones were handled made a noticeable difference too. Take away the tone-sensitive layer, and people felt less happy about outcomes by more than twelve points. In total, after two hundred users gave feedback, scores settled at 4.56 stars from five possible, just over ninety-one percent approval.

Putting it all together, the outcomes show Face2Frame moves past older models stuck on just one feature while backing up every key decision shaped during development. The evidence lines up clearly behind both progress and approach.

6.2 Limitations of the Work

Even so, some downsides exist. Though carefully selected and evenly distributed, the data pool holds just 1,500 face pictures - fewer than ideal when capturing every possible variation by race, age, or head tilt. For now, only near-front views work well. Side glances or half-turned faces fall outside what it can handle.

Warm light messes with how the system reads skin color by changing the b channel, sometimes moving results into a different group. Though there are many glasses styles listed, someone has to add them by hand, so nothing changes when new ones come out or tastes shift.

One step at a time, the Flask model handles tasks in order, leaving little room for multiple requests at once. In the end, testing leaned heavily on user feelings instead of actual buying results over time - data that could have better proven how good the suggestions really are.

6.3 Future Scope

Looking ahead, fresh paths pop up for growing Face2Frame past the early test version. Some routes stand out more than others when it comes to real progress. Moving forward might mean trying setups that haven't been tried yet. Each option brings something different to the table, depending on where effort lands next

Picture this. A live video view shows your face, then adds glasses frames right on top - like they are really there. These suggestions come from smart picks based on what suits you. The tech blends advice with sight, so you see how things look before deciding. It turns guessing into seeing, quietly, without fuss.

Putting the EfficientNet-B0 model into ONNX or TensorFlow Lite means it can run directly on phones. This works on both iPhones and Android devices without needing constant internet. Instead of sending data back and forth online, everything happens right on the device. Because of that, apps keep working even when there is no connection.

A larger mix of people in the data - different skin tones, ages, how faces are turned, and room brightness - helps the system work better for everyone. When more variety shows up during training, results across groups become more balanced. Seeing diverse real-world situations makes mistakes less likely in practice. Performance gaps between categories shrink when exposure is wider. Broader examples lead to steadier outcomes overall.

What users pick or rate after getting suggestions feeds back into the system. Over time, those choices tweak how scores are calculated - no fixed rules stay untouched. Learning happens quietly, shaped by real behavior instead of preset logic. Each adjustment leans on what people actually engage with, slowly shifting focus where it matters most.

Live video feeds could improve accuracy beyond what one still picture offers. Processing each frame as it arrives sharpens results instantly. Instead of relying on frozen moments, motion brings clearer data forward. Capturing expressions in real time removes guesswork tied to fixed angles. Continuous input refines measurements naturally. The system adjusts faster when fed streaming visuals. Immediate analysis trims delays found in batch handling.

Lighting changes mess with skin color detection. Instead of ignoring that, match the adjustment to whatever light fills the room. This tweak lines up better when lights lean orange or blue. Fixing for the main light source sharpens the result.

REFERENCES

- [1] M. Lin, Y. Wang, and Z. Liu, "A virtual eyeglass try-on system based on facial landmark detection for online eyewear shopping," *IEEE Access*, vol. 9, pp. 14201–14213, 2021.
- [2] J. Zhang, J. Li, and Q. Zhao, "Face shape classification using Inception-v3 deep learning architecture," *Multimedia Tools and Applications* (Springer), vol. 78, no. 6, pp. 7929–7944, 2019.
- [3] S. Patel, A. Mehta, and R. Kulkarni, "Intelligent eyewear recommendation system using machine learning," *Procedia Computer Science* (Elsevier), 2019.
- [4] J. Kim, S. Park, and H. Lee, "Real-time glasses try-on using augmented reality and deep learning," *Electronics* (MDPI), 2022.
- [5] Y. Wang, K. Lin, and S. Tang, "iGlasses: A novel recommendation system for best-fit glasses," in *Proc. ACM SIGIR*, 2019.
- [6] X. Li, Y. Ma, and Z. Chen, "3D parametric human face modeling for personalized product design: Eyeglasses frame design case," *Advanced Engineering Informatics*, vol. 39, 2021. <https://doi.org/10.1016/j.aei.2017.03.001>
- [7] P. G. González-Méijome, J. Jorge, and M. Queirós, "Comparison of physiognomy and frame angle parameters using different devices to prescribe progressive addition lenses," *Clinical and Experimental Optometry*, 2022. <https://doi.org/10.1080/08164622.2021.1914511>
- [8] S. Lee, J. Park, and M. Kim, "Integrated multi-model face shape and eye attributes identification for hairstyle and eyelashes recommendation," *Computation* (MDPI), vol. 9, no. 5, 2021. <https://doi.org/10.3390/computation9050054>
- [9] J. Ramon, B. Rossion, and A. Dricot, "The roles of shape and texture in the recognition of familiar faces," *Vision Research*, 2022. <https://doi.org/10.1016/j.visres.2022.108013>
- [10] A. Bulat and G. Tzimiropoulos, "A review of facial landmark extraction in 2D images and videos using deep learning," *Big Data and Cognitive Computing*, 2020. <https://doi.org/10.3390/bdcc3010014>
- [11] M. Basso, L. Mazzoli, and F. Bonarrigo, "Parametric design for custom-fit eyewear frames," *Heliyon*, 2023. <https://doi.org/10.1016/j.heliyon.2023.e19946>
- [12] S. B. G. Reddy, V. R. K. Reddy, and R. K. Khurana, "Head and facial anthropometry of the Indian population for designing a spectacle frame," *Indian Journal of Ophthalmology*, 2023. https://doi.org/10.4103/ijo.IJO_954_22
- [13] A. Sanna, F. Lamberti, and C. Demartini, "Designing an AI-based virtual try-on web application," *Sensors*, 2022. <https://doi.org/10.3390/s22103832>
- [14] Y. Zhou, H. Liu, and J. Wang, "A parametric product design framework for mass-customized face (eyewear) products," *Applied Sciences* (MDPI), 2021. <https://doi.org/10.3390/app11125382>

- [15] M. Al-Saffar, A. A. Abbood, and M. M. Al-Dabag, "Deep learning-based face shape classification system with application to eyeglass frame recommendation," *Applied Soft Computing*, 2025. <https://doi.org/10.1016/j.eswa.2025.129115>
- [16] Shrestha et al., "A web-based AR-powered virtual eyewear try-on system," *International Journal on Engineering Technology*, 2023.
- [17] T. Takahashi, Y. Kuroda, and H. Saito, "3D fitting simulation of eyeglass frames using inverse kinematics," *Journal of the Japan Society for Precision Engineering*, 2014.
- [18] C. H. Lin, Y. C. Tsai, and M. J. Wang, "A female glasses fitting model based on neural network," *Key Engineering Materials*, 2017.
- [19] A. M. Sorgato, L. Giannini, and A. Rizzi, "A qualitative study for parametric designed custom-fit eyewear frames," *Lecture Notes in Computer Science (Springer)*, 2024.
- [20] Y. Deng, J. Yang, and Z. Huang, "Automatic eyeglass virtual try-on system based on 3D face reconstruction," *Multimedia Tools and Applications (Springer)*, 2023.

APPENDICES

A.1 Face Detection and Preprocessing

A single image arrives, unprocessed, uploaded directly by the user. From that frame, faces emerge through OpenCV's Haar-based detection system. Once found, the face area expands slightly - by 20 percent around its edges - to keep parts like the forehead and jaw visible. When light levels fall too low, adjustments begin in the CIELAB space, specifically targeting the L-channel. There, CLAHE lifts contrast gently, avoiding harsh shifts. Afterward, gamma correction fine-tunes overall brightness while leaving colours untouched. A clean, resized version: exactly 224 by 224 pixels, full RGB, prepped ahead of classification

```
"""
```

```
Face shape classifier using MobileNetV2 (transfer learning).
```

```
Classes: Round, Oval, Square, Heart, Oblong.
```

```
Input: 224x224 RGB images.
```

```
Output: class label and optional probabilities.
```

```
"""
```

```
from pathlib import Path
```

```
from typing import Union
```

```
import numpy as np
```

```
# Try TensorFlow; fallback message if not installed
```

```
try:
```

```
    import tensorflow as tf
```

```
    from tensorflow import keras
```

```
    from tensorflow.keras import layers
```

```
    from tensorflow.keras.applications import MobileNetV2
```

```
    from tensorflow.keras.applications.mobilenet_v2 import prepro
```

```
except ImportError:
```

```
    tf = None
```

A.2 EfficientNet-B0 Hybrid Model Architecture

The classification module combines a pretrained EfficientNet-B0 CNN with a parallel branch that encodes five geometric facial ratios. EfficientNet-B0 is loaded with ImageNet weights and its final classification head is replaced by a global average pooling layer. The geometric ratio vector is normalised and passed through a fully connected layer. Both branches are concatenated and processed by two dense layers with ReLU activation and dropout regularisation before producing a five-class softmax output for face shape prediction.

```
class FaceFeatureExtractor:
    """
    Extracts facial features using MediaPipe Face Mesh.

    Usage:
        extractor = FaceFeatureExtractor()
        result = extractor.extract("face.jpg")
        print(result.to_dict())
    """

    def __init__(
        self,
        skin_tone_thresholds: Optional[SkinToneThresholds] = None,
        min_detection_confidence: float = 0.5,
        min_tracking_confidence: float = 0.5,
    ):
        """
        Initialize the face feature extractor.

        Args:
            skin_tone_thresholds: Custom thresholds for skin tone classi
            min_detection_confidence: Minimum confidence for face detect
            min_tracking_confidence: Minimum confidence for landmark tr
        """
        if not MEDIAPIPE_AVAILABLE:
            raise ImportError(
                "MediaPipe is required. Install with: pip install mediapipe"
            )

        self.skin_tone_thresholds = skin_tone_thresholds or SkinToneThresholds()
        self.min_detection_confidence = min_detection_confidence
        self.min_tracking_confidence = min_tracking_confidence
```

A.3 MediaPipe Landmark Extraction and Ratio Computation

This module uses MediaPipe's Face Landmarker to detect 478 anatomically consistent keypoints on the facial surface. From these landmarks, seven spatial measurements are derived - including jaw width, cheekbone span, forehead width, and face length. Five normalised geometric ratios are then computed from these measurements. These ratios serve as the structured input to the hybrid model's geometry branch and also support the prediction stability analysis module with confidence scores and accuracy.

```
# Facial Width Ratio → Frame Width Compatibility
# cheek to length_ratio thresholds
WIDTH_RATIO_RULES = {
    "narrow": (0.0, 0.70),    # Narrow face
    "medium": (0.70, 0.85),   # Medium face
    "wide": (0.85, 1.5),      # Wide face
}

# -----
# Data Classes
# -----

@dataclass
class GlassesFrame:
    """Represents a glasses frame from the catalog."""
    frame_id: str
    frame_shape: str
    frame_color: str
    rim_type: str
    frame_width_category: str
    image_path: str

    @classmethod
    def from_csv_row(cls, row: Dict[str, Any]) -> "GlassesFrame":
        """Create from CSV row dictionary. Handles pandas NaN via _str()."""
        def _str(v: Any) -> str:
            if v is None or (isinstance(v, float) and math.isnan(v)):
                return ""
            return str(v).strip()

        image_path = _str(row.get("image_path")) or "/static/glasses/placeholder"
        if image_path and not image_path.startswith("/"):
            image_path = "/static/glasses/" + image_path.lstrip("/")
        return cls(
            frame_id=_str(row.get("frame_id")),
            frame_shape=_str(row.get("frame_shape")).lower(),
            frame_color=_str(row.get("frame_color")).lower(),
            rim_type=_str(row.get("rim_type")).lower(),
            frame_width_category=_str(row.get("frame_width_category")) or "medium",
            image_path=image_path,
        )
```

A.4 ITA Skin Tone Detection

From the adjusted face picture, a spot on the cheek is taken, carefully missing shiny areas like the nose ridge or brow. This area shifts from BGR into CIELAB colour format, where average readings from the L and b channels help find the Individual Typology Angle. Out comes an ITA number that slots the person into Fair, Medium, or Dark skin range, guiding how frame shades get narrowed down later.

```
# -----  
# Skin Tone Classification  
# -----  
  
class SkinTone(Enum):  
    """Skin tone classification."""  
    FAIR = "fair"  
    MEDIUM = "medium"  
    DUSKY = "dusky"  
    DARK = "dark"  
    UNKNOWN = "unknown"  
  
@dataclass  
class SkinToneThresholds:  
    """  
    Configurable thresholds for skin tone classification.  
    Based on V (Value/Brightness) channel in HSV color space.  
  
    V channel range: 0-255  
    - Higher V = lighter skin  
    - Lower V = darker skin  
    """  
    fair_min: int = 180    # V >= 180 = Fair  
    medium_min: int = 130  # V >= 130 = Medium  
    dusky_min: int = 80    # V >= 80 = Dusky  
    # V < 80 = Dark  
  
    # -----
```

A.5 Recommendation Engine Scoring Logic

Once it gets the forecasted face shape, skin tone group, and measurement ratios, the system begins. First comes a rule-driven check matching shapes. Then a second layer evaluates colour harmony using the user's complexion type. After that, an assessment of proportions takes place based on spatial features. Each item in the database earns a combined rating through scaled blending of these three results. Out of those ratings, five frame options rise to the top. With them come plain-English reasons explaining why they fit.

```

# Face Shape → Recommended Frame Shapes (with compatibility scores 0-1)
FACE_SHAPE_RULES: Dict[str, Dict[str, float]] = {
    "round": {
        # Angular frames to add definition
        "rectangular": 1.0,
        "square": 0.9,
        "wayfarer": 0.85,
        "browline": 0.8,
        "geometric": 0.75,
        "cat-eye": 0.6,
        "aviator": 0.5,
        # Avoid round frames (too similar)
        "round": 0.2,
        "oval": 0.3,
    },
    "square": {
        # Round/oval frames to soften angles
        "round": 1.0,
        "oval": 0.95,
        "aviator": 0.85,
        "cat-eye": 0.7,
        "browline": 0.6,
        # Avoid angular frames
        "rectangular": 0.3,
        "square": 0.2,
        "geometric": 0.4,
        "wayfarer": 0.5,
    },
    "oval": {
        # Versatile - most frames work
        "rectangular": 0.9,
        "square": 0.85,
        "round": 0.85,
        "oval": 0.8,
        "wayfarer": 0.9,
        "aviator": 0.85,
        "cat-eye": 0.8,
        "browline": 0.85,
        "geometric": 0.8,
    },
    "heart": {
        # Bottom-heavy frames to balance wide forehead
        "aviator": 1.0,
        "round": 0.85,
        "oval": 0.8,
        "wayfarer": 0.75,
    }
}

```


Face2Frame: Intelligent Eyeglass Frame Recommendation Using Facial Analysis

ORIGINALITY REPORT

7%

SIMILARITY INDEX

4%

INTERNET SOURCES

1%

PUBLICATIONS

5%

STUDENT PAPERS

PRIMARY SOURCES

1

Submitted to VNR Vignana Jyothi Institute of Engineering and Technology

Student Paper

4%

2

Submitted to Jawaharlal Nehru Technological University Kakinada

Student Paper

1%

3

digitallibrary.mes.ac.in

Internet Source

<1%

4

ir.library.nitw.ac.in:8080

Internet Source

<1%

5

meconthtacet.web.app

Internet Source

<1%

6

ir.juit.ac.in:8080

Internet Source

<1%

7

5dok.net

Internet Source

<1%

8

www.coursehero.com

Internet Source

<1%

9

dspace.daffodilvarsity.edu.bd:8080

Internet Source

<1 %

10

tudr.thapar.edu

Internet Source

<1 %

11

ijtsm.ilmauniversity.edu.pk

Internet Source

<1 %

12

Submitted to APJ Abdul Kalam Technological University, Thiruvananthapuram

Student Paper

<1 %

13

Submitted to University of Hong Kong

Student Paper

<1 %

14

Submitted to Asia Pacific University College of Technology and Innovation (UCTI)

Student Paper

<1 %

15

comphy.tistory.com

Internet Source

<1 %

16

hpcc.umd.edu

Internet Source

<1 %

17

www.ir.juit.ac.in:8080

Internet Source

<1 %

18

objectstorage.ap-dcc-gazipur-1.oraclecloud15.com

Internet Source

<1 %

19

ir.uitm.edu.my

Internet Source

<1 %

nepjol.info

Paolo Ferro, Harinadh Vemanaboina, Chander Prakash. "Computational Techniques and Smart Manufacturing", CRC Press, 2026
Publication

Exclude quotes On
Exclude bibliography On

Exclude matches < 6 words